CS5785 / ORIE5750 / ECE5414 Homework 3

This homework is due on **Monday, October 19, 2026 at 11:59PM ET**. It consists of a Canvas quiz component, programming exercises, and written exercises. Please complete the quiz on Canvas, and upload your solutions to the programming and written exercises to Gradescope.

Homework 3 consists of three parts:

1. A Canvas survey titled "HW3: Quiz" found in the "Assignments" tab, to check your understanding of the prerequisite material required for this course.
2. A PDF write-up used to complete the "Written Exercises" section of this homework. Upload your PDF write-up solution to Gradescope, under the assignment title "Homework 3 - Report".
3. Source code and data files used to complete the "Programming Exercises" section of this homework. Code/data for all of your experiments, and figures if required, should be present in `.ipynb` files. These files should be placed in a folder titled `hw3`, compressed as a .zip file, and uploaded to the "Homework 3 - Code" assignment in Gradescope.

The write-up should contain a general summary of what you did, how well your solution works, any insights you found, etc. On the cover page, include the class name, and homework number. You are responsible for submitting clear, organized answers to the questions. You could use online LaTeX templates from Overleaf, under “Homework Assignment” and “Project / Lab Report”.

Please include all relevant information for a question, including text response, equations, figures, graphs, output, etc. If you include graphs, be sure to include the source code that generated them in your code submission to "Homework 3 - Code" assignment in Gradescope.

To ease graders’ job, please **tag your homework questions** appropriately on Gradescope. Failure to do so will result in a small point deduction. Please also **include the write-up of your coding exercise** in your writeup. You could either directly write your entire homework in Jupyter Notebook, or provide screenshots of your code and your output along with your written answers.

IF YOU NEED HELP

There are several strategies available to you.

- If you get stuck, we encourage you to post a question on ED Discussion. That way, your solutions will be available to other students in the class.
- The professor and TAs offer office hours, which are a great way to get some one-on-one help.
- You are allowed to use well known libraries such as `scikit-learn`, `scikit-image`, `numpy`, `scipy`, etc. for this assignment (including implementations of machine learning algorithms), unless we explicitly say that you cannot in a particular question. Any reference or copy of public code repositories should be properly cited in your submission (examples include Github, Wikipedia, Blogs).

PROGRAMMING EXERCISES

1. **Binary Classification of Clinical-Trial Text.** In this problem, you will use the Naive Bayes rule from class to perform classification on text data.

   We will continue working with the clinical-trial data from HW2, where `label=1` denotes success and `label=0` denotes failure. In this question, combine the Phase I, Phase II, and Phase III data and include ICD codes in addition to the eligibility criteria, disease names, and drug names. You may reuse your HW2 preprocessing code. Train each model once on the combined data, rather than separately for each phase.

   Download the data.

   Download the updated files from the course Kaggle competition:

   `phase1_train.csv`, `phase1_valid.csv`,
   `phase2_train.csv`, `phase2_valid.csv`,
   `phase3_train.csv`, and `phase3_valid.csv`.

   These are the HW2 datasets with an additional `icdcodes` column.

   Combine the provided splits. Concatenate the three training files into one training set and the three validation files into one validation set. Some trials occur in the training set of one phase and the validation set of another. Remove from the combined training set every row whose `NCTID` also occurs in the combined validation set. Keep the validation set unchanged. Use these two sets throughout this question; do not randomly re-split the data. This exercise is evaluated on training and validation data and does not require a new Kaggle submission. The test files and the HW2 submission format remain unchanged.

   Preprocess the data.

   For each trial, concatenate `criteria`, `diseases_cleaned`, `drugs`, and `icdcodes`, separated by spaces, into one text string. You may reuse and adapt your HW2 preprocessing code.

   ICD codes are standardized identifiers for diseases and related conditions, rather than ordinary English words. They contain letters and digits and may include a decimal point, as in `R11.0`. The disease, drug, and ICD fields may contain lists stored as strings, including nested list strings. Keep these formats in mind when choosing your preprocessing: for example, removing digits or splitting at punctuation can change an ICD code. No medical knowledge or lookup of code descriptions is required. You **may or may not** want to do one or all of the following. Make sure to note how you preprocess the data. Explain the reasons for your choices, including any differences in how you process ordinary text and ICD codes.

   - Convert all the words to lowercase.
   - Lemmatize or stem words.
   - Strip punctuation.
   - Strip the stop words, e.g., *the, and, or*.
   - Remove or retain numbers and structural terms such as *inclusion* and *exclusion*.

- Remove brackets, quotation marks, and other list-formatting symbols from the disease, drug, and ICD fields.
- Choose how to tokenize ICD codes, for example, by preserving the decimal point with a suitable tokenizer or removing it to keep the letters and digits together. You may also add a prefix such as `icd_` to distinguish code tokens from other text.
- Something else? Tell us about it.

Explain how you handle missing or empty values, and apply the same preprocessing procedure to the training and validation sets.

Bag of words model. The next task is to extract features in order to represent each trial using the binary bag-of-words model, as discussed in lectures. The idea is to build a vocabulary of the words appearing in the training set, and then to represent each trial by a feature vector x whose length is the same as the size of the vocabulary, where $x_i = 1$ if the ith vocabulary token appears in that trial, and $x_i = 0$ otherwise. In order to build the vocabulary, you should choose some threshold M, and only include tokens that appear in at least M different training trials; this is important both to avoid run-time and memory issues, and to avoid noisy/unreliable features that can hurt learning. Decide on an appropriate threshold M, then build the bag-of-words feature vectors for both the training and validation sets. Explain your choice of M and report the vocabulary size. Use binary unigrams in this question.

In order to construct these features, we suggest using the CountVectorizer class in `sklearn`. Set `binary=True` to ensure that the feature vectors are binary, and use `min_df=M` to include only tokens that appear in at least M different training trials. Set `ngram_range=(1,1)`. Fit CountVectorizer only once on your training set and use the same instance to process both your training and validation sets; do not refit it on the validation set.

Important: Both parts below use the same feature matrices. Use only these four fields as predictive inputs: `criteria`, `diseases_cleaned`, `drugs`, and `icdcodes`. Do not use `NCTID` or `phase` as predictive features.

Evaluation. As in HW2, report F1, ROC-AUC, and PR-AUC on both the training and validation sets for each classifier. Compute F1 from binary predictions, with `label=1` as the positive class and a probability threshold of 0.5. Compute ROC-AUC and PR-AUC from the predicted probabilities for `label=1`. You may use the following functions from `sklearn.metrics`:

`f1_score`, `roc_auc_score`, and `average_precision_score`.

As in HW2, report average precision as PR-AUC. Use validation F1 to select the best model; discuss the other metrics as supporting results.

(a) **Bernoulli Naive Bayes.** Implement a Bernoulli Naive Bayes classifier using Laplace smoothing to predict the probability of whether a trial has `label=1`. Train this classifier on the training set, and report its training and validation F1, ROC-AUC, and PR-AUC.

Important: For this question, you should implement the Naive Bayes classifier yourself, similar to what was shown in class, without using an existing machine-learning implementation such as `sklearn.naive_bayes`. You may use basic libraries such as `numpy`. This restriction applies only to the Naive Bayes classifier; the vectorization and evaluation functions allowed elsewhere in the question may still be used.

As you work on this problem, you may find that there exist some features that only appear in some classes but never in other classes in the training set. This might lead the model into thinking it is impossible for the feature to appear in those other classes, even though it might actually appear in those classes in the validation set.

Concretely, suppose feature j never appears in class k in the training set. Since the maximum likelihood estimate of ψ_{jk} is $\frac{n_{jk}}{n_k}$, where n_k is the number of documents of class k and n_{jk} is the number of documents of class k that contain feature j, during training we set

$$P(x_j = 1 \mid y = k) = \psi_{jk} = \frac{n_{jk}}{n_k} = 0.$$

Then, any bag of words x that contains feature j, i.e., $x_j = 1$, will have

$$P_\theta(x \mid y = k) = \prod_{j=1}^{d} P(x_j \mid y = k) = 0.$$

We do not want this, because it implies we only assign positive probability to bags of words whose features have appeared in class k at least once.

The solution to this problem is a form of regularization called Laplace smoothing or additive smoothing. The idea is to use pseudo-counts, incrementing the number of class k documents containing feature j by α virtual occurrences and the total number of class k documents by 2α (half of the pseudo documents contain feature j). Mathematically,

$$\psi_{jk} = \frac{n_{jk} + \alpha}{n_k + 2\alpha}.$$

Use $\alpha = 1$; no search over smoothing parameters is required. Estimate the class priors from the training labels and account for both present and absent features. Use log probabilities to avoid numerical underflow and normalize the class scores to obtain posterior probabilities for ROC-AUC and PR-AUC.

(b) **Model comparison.** You just implemented a generative classifier, and in HW2 you implemented a discriminative classifier (logistic regression). Since the data are now combined across phases and include ICD codes, retrain logistic regression on the same feature matrices used in part (a). As in HW2, fit three versions: no regularization, L1 regularization, and L2 regularization. You may use `sklearn`'s LogisticRegression, with `penalty=None`, `'l1'`, or `'l2'`, respectively, and a solver compatible with the chosen penalty. Report training and validation F1, ROC-AUC, and PR-AUC for all three versions. For the L1 and L2 models, you may use `C=1.0`; no additional hyperparameter search is required. Reflect on the following:

- Which logistic regression version achieves the highest validation F1? Compare it with Bernoulli Naive Bayes from part (a), and state which classifier you would select based on validation F1. Do ROC-AUC and PR-AUC support the same conclusion? Comment on the pros and cons of using generative versus discriminative models.
- Think about the assumptions that Naive Bayes makes. How are the assumptions different from logistic regression? Discuss whether it is valid and efficient to use a Bernoulli Naive Bayes classifier for natural language texts.

2. **Gaussian Discriminant Analysis** In this problem, you will implement a generative model for multiclass classication using a GDA approach. 'Generative' in this context means that instead of directly computing the likliehood of each class y for each sample x, our approach instead involves computing the prior probability of a class y and the likelihood that a sample x is generated from a class y.

   You will need to use the Iris Flower Data set. The data file contains 150 observations of three different flower types (*Iris Setosa*, *Iris Versicolour*, *Iris Virginica*) and four different measurements of the size of different parts of the flower (sepal length, sepal width, petal length, petal width). Note that the dataset can be loaded using the following code:

```
from sklearn import datasets
iris = datasets.load_iris(as_frame=True)
```

   Split the Iris Flower Dataset into test and training sets, and use only the training set for questions (a)-(c) and (e). For question (d), use the test set.

   You should do this without calling the Discriminant Analysis library in `scikit learn`. You can use `numpy` or `scipy` for matrix calculation or generating Gaussian distributions. Additionally, we provide a tutorial, on a subset of the Iris Flower Data set, of how to run GDA. See the code companion notebook (titled HW3 Code Companion) on Canvas.

   (a) Select any two columns from the four features in the dataset. Treat each data entry as a 2-dimensional feature vector. Parse and plot all data points on a 2-D plane - using a different color for each class, label each data point according to what flower class it belongs to. Comment on any patterns you notice between feature clustering and flower classes. Note that for the rest of the problem, you will use all four features.

   (b) Recover and print the prior probability of each class, i.e. for each class k, find the proportion of observations in that class: $\frac{n_k}{n}$.

   (c) Assuming that each class corresponds to a different multivariate Gaussian, compute the empirircal mean and empirical covariance matrix for each class, i.e., for each class k compute μ_k, Σ_k and print the values.

   (d) Implement and run GDA to classify the flowers in your test set. Specifically:

      i. Use the prior probability of each class calculated in part (b) and the Gaussian distribution parameters calculated in part (c) to calculate for each sample x the class y with the highest probability. In other words, use (b) to calculate $P(y)$, (c) to calculate $P(x|y)$, and put them together to obtain

$$\arg\max_y P(y \mid x) = \arg\max_y \left[P(x \mid y) P(y) \right]$$

      ii. Report the results of your classifier using whatever metric you think is most appropriate. Explain and justify your choice of metric.

   (e) If you were to run K-means clustering instead of the GDA algorithm you just implemented, do you think you would get different clusters? Experiment with K-means (with $K = 2$) clustering on the same subset of the dataset used in part (a). (The K-Means library from `scikit-learn` is a good way to try). Comment on why you think the results will or will not change.

WRITTEN EXERCISES

1. **Naive Bayes with Binary Features.** Consider a group of 50 Cornell Students. 20 of them are Master's students, while the rest 30 of them are PhD students. There are 5 Master's students who bike, and there are 5 Master's students who ski. On the other hand, 20 PhD students bike, and 15 PhD students ski.

   We can formulate this as a machine learning problem by modeling the students with features $x = (x_1, x_2) \in \{0,1\}^2$, where x_1 is a binary indicator of whether the students bike and x_2 is a binary indicator of whether they ski, and the target y equals 1 if they are PhD students and 0 if they are Master's students.

   (a) Please elaborate in this context what is the Naive Bayes assumption.

   (b) With the Naive Bayes assumption, (probability of biking and skiing are conditionally independent given a study program), find the probability of a student in this group who neither bikes or skis being a Master's student

   (c) Suppose we know that every PhD who skis also bikes. Does it make sense to still assume that probability of biking and skiing are conditionally independent for a PhD student? If not, how would your answer to part (b) change with this knowledge (you can still assume probability of biking and skiing are conditionally independent for a Master's student)?

2. **Categorical Naive Bayes.** Suppose we are working with a dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)}) \mid y = 1, 2, ..., n\}$ in which the d-dimensional inputs x are *categorical*: each feature x_j takes one of L possible values: $x_j^{(i)} \in \{1, 2, ..., L\}$ for all i, j. If $L = 2$, then the features look like the binary bag-of-words vectors that we have seen in class; in this example, however, the features can take more than just two values. We also assume that the target y represents one of K possible classes: $y \in \{1, 2, ..., K\}$

   In the Categorical Naive Bayes algorithm, we model this data via a probabilistic model $P_\theta(x, y)$.

   - The distribution $P_\theta(y)$ is Categorical with parameters $\phi = (\phi_1, ..., \phi_K)$ and

     $$P_\theta(y = k) = \phi_k$$

   - The distribution of each feature x_j conditioned on $y = k$ is a Categorical distribution with parameters $\psi_{jk} = (\psi_{jk1}, ..., \psi_{jkL})$, where

     $$P_\theta(x_j = \ell | y = k) = \psi_{jk\ell}.$$

     The distribution over a vector of features x is given by

     $$P_\theta(x | y = k) = \prod_{j=1}^{d} P_\theta(x_j | y = k),$$

     which is just the Naive Bayes factorization of $P_\theta(x | y = k)$.

   In other words, the prior distribution $P_\theta(y)$ in this model is the same as in Bernoulli Naive Bayes. The distribution $P_\theta(x | y = k)$ is a product of Categorical distributions, whereas in Bernoulli Naive Bayes it was the product of Bernoulli distributions.

The total set of parameters of this model is $\theta = (\phi_1, ...\phi_K, \psi_{111}, ...\psi_{dKL})$. We learn the parameters via maximum likelihood: $\max_\theta \frac{1}{n}\sum_{i=1}^{n} \log P_\theta(x^{(i)}, y^{(i)})$

(a) Show that the maximum likelihood estimate for the parameters ϕ_k is

$$\phi_k^* = \frac{n_k}{n},$$

where n_k is the number of data points with class k.

(b) Show that the maximum likelihood estimate for the parameters $\psi_{jk\ell}$ is

$$\psi_{jk\ell}^* = \frac{n_{jk\ell}}{n_k},$$

where $n_{jk\ell}$ is the number of data points with class k for which the j-th feature equals ℓ.